Automaten und Sprachen | AutoSpr

Zusammenfassung

INHALTSVERZEICHNIS

1. Prädikate **2**
1.1. Normalformen 2
1.2. Negation 2
2. Mengen **2**
2.1. Konstruktion 2
2.2. Operationen 2
2.3. Tupel 2
3. Beweise **2**
4. Sprachen **3**
4.1. Wortlänge 3
4.2. Deterministische Endliche Automaten (DEA) 4
4.3. Wörter einer regulären Sprache 4
4.3.1. Übergangsfunktionen für Wörter 4
4.3.2. Wort akzeptieren 4
4.3.3. Akzeptierte Sprache, reguläre Sprache 4
4.4. Myhill-Nerode Automat 5
4.5. Minimalautomat 5
4.6. Pumping Lemma 5
4.6.1. Beweis 6
4.7. Nichtdeterministische endliche Automaten (NEA) 6
4.7.1. Umgang mit mehrdeutigen Übergängen 7
4.7.2. Thompson-NEA 7
4.7.3. $\mathbf{NEA}_{\varepsilon}$ 7
4.8. Mengenoperationen 7
4.8.1. Produkt 7
4.8.2. Schnittmenge 8
4.8.3. Vereinigung 8
4.8.4. Differenz 8

1. PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionen» mit booleschen Rückgabewerten: $P, Q(n), R(x, y, z)$. Siehe DMI .

1.1. NORMALFORMEN

Normalformen (allgemein, ***kanonische Formen***) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \ldots \wedge P_n = \forall i \in \{1, \ldots, n\}(P_i) = \bigwedge_{i=1}^{n} P_i$$

$$P_1 \vee \ldots \vee P_n = \exists i \in \{1, \ldots, n\}(P_i) = \bigvee_{i=1}^{n} P_i$$

1.2. NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht

$$\neg\forall i \in \{1, \ldots, n\}(P_i) \Leftrightarrow \neg(P_1 \wedge \ldots \wedge P_n) \Leftrightarrow \neg P_1 \vee \ldots \vee \neg P_n \Leftrightarrow \exists i \in \{1, \ldots, n\}(\neg P_i)$$

2. MENGEN

2.1. KONSTRUKTION

Grundmenge G, Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

2.2. OPERATIONEN

Begriff	***Bedeutung***
Vereinigung	$A \cup B = \{x \mid x \in A \vee x \in B\}$
Schnittmenge	$A \cap B = \{x \mid x \in A \wedge x \in B\}$
Komplement	$\overline{A} = \{x \mid x \notin A\}$
Differenz	$A \setminus B = \{x \in A \mid x \notin B\}$

2.3. TUPEL

$$A \times B = \{(a, b) \mid a \in A \wedge bnB\}$$

n-Tupel:

$$\bigtimes_{i=1}^{n} A_i = \{(a_1, a_2, \ldots, a_n) | a_i \in A_i\}$$

3. BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

Beweistechnik	***Anwendungsfall***
Konstruktiver Beweis	Liefert explizite «Lösung» / Algorithmus
Widerspruchsbeweis	Geeignet für Unmöglichkeitsaussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$
Vollständige Induktion	Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

4. SPRACHEN

Begriff	***Bedeutung***
Alphabet	Eine nichtleere endliche menge Σ heisst ***Alphabet***. Die Elemente von Σ heissen ***Zeichen***.
Wort	Eine Zeichenkette der Länge n ist ein n-Tupel in $\Sigma^n = \Sigma \times \ldots \times \Sigma$. Ein Element von Σ^n heisst ***Wort*** der Länge n. Wörter haben immer endliche Länge.
Leeres Wort	Die Zeichenkette $\varepsilon \in \Sigma^0 = \{\varepsilon\}$ der Länge 0 heisst das ***leere Wort***.
Menge aller Wörter	Leeres Wort ist **immer** drin $$\Sigma^* = \{\varepsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \ldots = \bigcup_{k=0}^{\infty} \Sigma^k$$
Abgekürzte Schreibweisen von Wörtern	$(A, u, t, o, S, p, r) = AutoSpr$ $a^3b^5 = aaabbbbb$ $b^5a^3 = bbbbbaaa$
Sprache	Teilmenge $L \subset \Sigma^*$

4.1. WORTLÄNGE

Begriff	***Bedeutung***
Länge des Wortes	$w \in \Sigma^n \Rightarrow \lvert w \rvert = n$, n ist ***Länge des Wortes*** w
Anzahl Zeichen	Sei $w \in \Sigma^n, a \in \Sigma$. Dann ist $\lvert w \rvert_a$ die ***Anzahl Zeichen*** a im Wort w

Beispiel 1 : **Wortlänge**

$$\lvert \varepsilon \rvert = 0 \qquad \lvert 01010 \rvert_1 = 2 \qquad \lvert a^3b^5 \rvert = 8$$
$$\lvert 01010 \rvert_0 = 3 \qquad \lvert 01010 \rvert_3 = 0 \qquad \lvert (1291)^7 \rvert = 7\lvert 1291 \rvert = 7 \cdot 4 = 28$$

Beispiel 2 : **Sprache**

$$L = \Sigma^* \subset \Sigma^*$$
$$L = \varnothing \subset \Sigma^*, \text{die leere Sprache}$$
$$\Sigma = \{0, 1\}, \text{Sprache aller Binärstrings: } L = \Sigma^*$$
$$\Sigma = \text{Unicode}, J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\}$$
$$M \text{ eine "Maschine"}, L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}$$

Beobachtungen 1

1.1. $\lvert w^n \rvert = n \cdot \lvert w \rvert$

1.2. Zu jeder Maschine M gibt es eine Sprache $L(M)$

4.2. DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

Definition

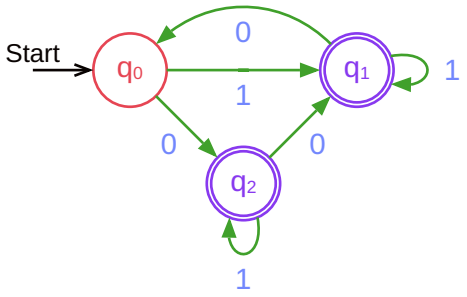
$$A = (Q, \Sigma, \delta, q_0, F)$$

- Zustände: $Q = \{q_0, q_1, \ldots, q_n\}$
- Alphabet: Σ
- Übergangsfunktion: $\delta : Q \times \Sigma \to Q$
- Startzustand: $q_0 \in Q$
- Akzeptierzustände: $F \subset Q$

Tabellenform

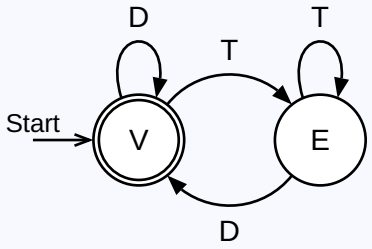
		0	1	Σ
Q	q_0	q_2	q_1	
	q_1	q_0	q_1	δ
F	q_2	q_1	q_2	

Zustandsdiagramm von A



Beispiel 3 : **Skipiste**

- Mögliche Aktionen: T, D
- Zustände: V, E
- Startzustand: V
- Akzeptable «Endzustände»: $\{V\}$
- Übergänge



Wichtig:
- Ein Pfeil für jedes Zeichen in jedem Zustand (deterministischer endlicher Automat / DEA)

4.3. WÖRTER EINER REGULÄREN SPRACHE

4.3.1. Übergangsfunktionen für Wörter

$$\delta : Q \times E^* \to Q : (q, w = a_1, \ldots, a_n) \mapsto \delta(\ldots\delta(\delta(q, a_1), a_2), \ldots, a_n)$$

Übergänge ausghend von q für Zeichen $a_1, \ldots, a_n$ nacheinander anwenden.

4.3.2. Wort akzeptieren

Der DEA $A = (\Sigma, Q, q_0, \delta, F)$ ***akzeptiert*** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

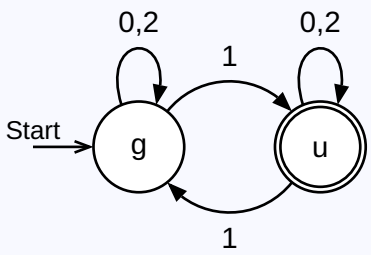
4.3.3. Akzeptierte Sprache, reguläre Sprache

Gegeben ein DEA $A = (\Sigma, Q, q_0, \delta, F)$. Die von A ***akzeptierte Sprache*** ist

$$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$$

Die Sprache $L \subset \Sigma^*$ heisst ***regulär***, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 4 : **DEA, der die Sprache $L = \{w \in \Sigma^* \mid w$ ist eine ungerade Zahl im Dreiersystem$\}$ akzeptiert**



	0	1	2
g	g	u	g
u	u	g	u

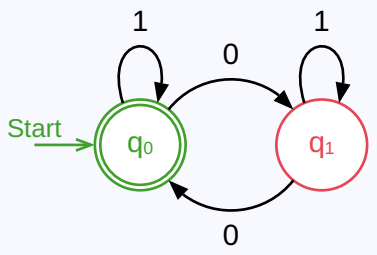
4.4. MYHILL-NERODE AUTOMAT

Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$. Insbesondere $L(\varepsilon) = L$

Gegeben: reguläre Sprache L über Σ. Rekonstruiere A mit:
- $Q = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\sigma) = L$
- $F = \{L(w) \in Q \mid \varepsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand ⇔ gleiches $L(w)$

Beispiel 5 : $\Sigma = \{0,1\}, L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$



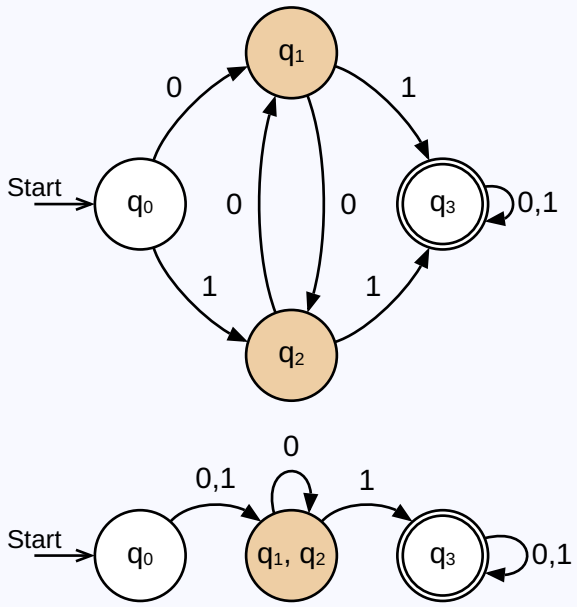
w	$L(w)$	Q
ε	$L(\varepsilon) = L$	q_0
0	$L(0) = \{w \in \Sigma^* \mid \lvert w\rvert_0 \text{ ungerade}\}$	q_1
1	$L(0) = \{w \in \Sigma^* \mid \lvert w\rvert_0 \text{ gerade}\}$	q_0
⋮	⋮	⋮

4.5. MINIMALAUTOMAT

Ziel: Nicht unterscheidbare Zustände zusammenlegen

Vorgehen:
1) Akzeptierzustand ≠ Nichtakzeptierzustand
2) Iteration: alle unterscheidbaren Paare suchen
3) Verbleibende Paare sind ununterscheidbar → zusammenlegen

Beispiel 6



	q_0	q_1	q_2	q_3
q_0	≡	×	×	×
q_1	×	≡	≡	×
q_2	×	≡	≡	×
q_3	×	×	×	≡

Algorithmus

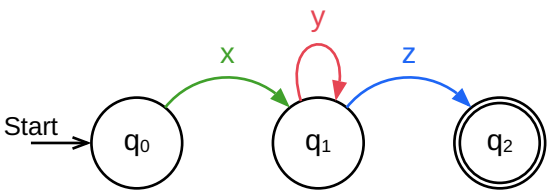
1) $q \in F, q' \notin F$
2) Unterscheidbar nach Übergang
3) Iteration bis keine Änderung mehr

⇒ q_1 und q_2 sind nicht unterscheidbar

4.6. PUMPING LEMMA

Ist L eine reguläre Sprache, dann gibt es $N \in \mathbb{N}$, die pumping length so, dass jedes Wort $w \in L$ mit $|w| \geq N$ in drei Teile $w = xyz$ zerlegt werden kann mit:

1) $|xy| \leq N$
2) $|y| > 0$
3) $xy^kz \in L \forall k \in \mathbb{N}$



Oder: genügend lange Wörter ($|w| \geq N$) einer regulären Sprache ($w \in L$) können alle in einem Anfangsstück der Länge N ($|xy| \leq N$) aufgepumpt werden ($xy^kz \in L$).

4.6.1. Beweis

Behauptung: Die Sprache $L \subset \Sigma^*$ ist nicht regulär.

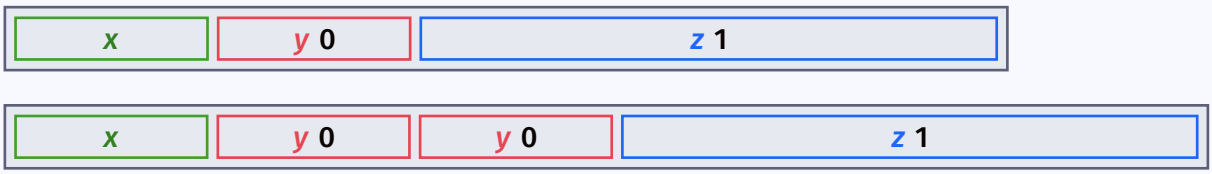
Beweis (Widerspruch):

1) Annahme: L ist regulär
2) Gemäss Pumping Lemma gibt es die Pumping Length N
   – Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
3) Wähle ein Wort $w \in L$ mit $|w| \geq N$
   – Die Definition **muss** N verwenden!
4) Aufteilung des Wortes gemäss Pumping Lemma
   – $w = xyz, |xy| \leq N, |y| > 0$
5) Auswirkung des Pumpens
   – $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 7 : **$L = \{0^n1^n \mid n \geq 0\}$ ist nicht regulär**

1) Annahme: L ist regulär
2) $\exists N \in \mathbb{N}$, Pumping Length
3) $w = 0^N1^N$
4) Unterteilung $w = xyz$

TODO:
finish



5) Pumpen: nur die Anzahl der **1** wird erhöhr, Anzahl **1** bleibt

– $xy^kz \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

4.7. NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

Definition

$$A = (Q, \Sigma, \delta, q_0, F)$$

– Endliche Menge von Zuständen: Q
– Alphabet: Σ
– Übergangsfunktion: $\delta : Q \times \Sigma \to P(Q)$
– Startzustand: $q_0 \in Q$

Akzeptieren

Ein NEA A akzeptiert das Wort $w \in \Sigma^*$, wenn es eine **Wahl** von Übergängen gibt, derart, dass das Wort w den Automaten in einen Akzeptierzustand überführt.

Faustregeln

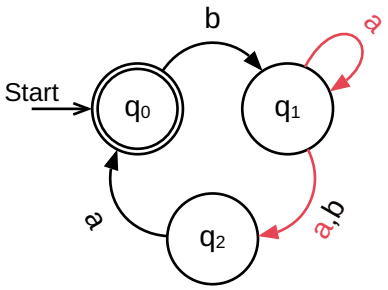
– Akzeptierzustände: $F \subset Q$

– Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.
– Erlaube Alternativen

Beispiel 8

TODO:

4.7.1. Umgang mit mehrdeutigen Übergängen



a-Übergang von q_1 aus nicht eindeutig

Welchen Übergang soll man nehmen?

– Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.

4.7.2. Thompson-NEA

TODO:
Thompson-NEA

4.7.2.1. Transformation NEA → DEA

TODO:

Gegeben $\delta : Q \times \Sigma \to P(Q)$ eines NEA. Übergangsfunktion für Mengen $M \subset Q$

$$\delta' : P(Q) \times \Sigma \to P(Q) : (M, a) \mapsto \delta'(M, a) = \bigcup_{q \in M} \delta(q, a)$$

Satz

Ein NEA A kann in einen DEA A' umgewandelt werden mit
– $Q' = P(Q)$
– $\Sigma' = \Sigma$
– δ' wie oben definiert
– $q'_0 = \{q_0\}$
– $F' = \{M \in P(Q) \mid F \cap M \neq \emptyset\}$

Implementation

Thompson-NEA:
– Zustand $M \subset Q$ realisiert durch Markierung der Zustände in M
– δ' realisiert durch Verschieben von Markierungen
– Akzeptieren: mindestens ein Akzeptierzustand markiert

4.7.3. NEA$_\varepsilon$

TODO:

ε –Übergang

4.8. MENGENOPERATIONEN

TODO:

Lassen reguläre Sprachen regulär sein.

4.8.1. Produkt

Verändert nur Endzustände

4.8.2. Schnittmenge

Beide Automaten akzeptieren.

4.8.3. Vereinigung

4.8.4. Differenz